J. M. Willman, *Beginning PyQt*
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_9

9. Graphics and Animation in PyQt

Joshua M. Willman[1]
(1) Hampton, VA, USA

After going through many examples in previous chapters that introduce you to the fundamentals for building GUIs, Chapter 9 finally allows for you to explore your creative and artistic side through drawing and animation in PyQt5.

Graphics in PyQt is done primarily with the **QPainter** API. PyQt's painting system handles drawing for text, images, and vector graphics and can be done on a variety of surfaces, including QImage, QWidget, and QPrinter. With QPainter you can enhance the look of existing widgets or create your own.

The main components of the painting system in PyQt are the **QPainter**, **QPaintDevice**, and **QPaintEngine** classes. QPainter performs the drawing operations; a QPaintDevice is an abstraction of two-dimensional space which acts as the surface that QPainter can paint on; and QPaintEngine is the internal interface used by the QPainter and QPaintDevice classes for drawing.

In this chapter, we are going to be taking a look at 2D graphics, covering topics such as drawing simple lines and shapes, designing your own painting application, and animation. If you are interested in creating GUIs that work with 3D visuals, Qt also has support for OpenGL, which is software that renders both 2D and 3D graphics.

New concepts introduced in this chapter include

- An introduction to `QPainter` and other classes used for drawing PyQt
- Creating tool tips using `QToolTip`
- Animating objects using `QPropertyAnimation` and `pyqtProperty`
- How to set up a Graphics View and a Graphics Scene for interacting with items in a GUI window
- A new widget for selecting values in a bounded range, `QSlider`
- Handling mouse events with event handlers
- PyQt's four image handling classes
- Creating your own custom widgets using PyQt

Introduction to the QPainter Class

Whenever you need to draw something in PyQt, you will more than likely need to work with the QPainter class. QPainter provides functions for

drawing simple points and lines, complex shapes, text, and pixmaps. We have looked at pixmaps in previous chapters in applications where we needed to display images. QPainter also allows you to customize a variety of its settings, such as rendering quality or changing the painter's coordinate system. Drawing can be done on a **paint device**, which are two-dimensional objects created from the different PyQt classes that can be painted on with QPainter.

Drawing relies on a coordinate system for specifying the position of points and shapes and is typically handled in the paint event of a widget. The default coordinate system for a paint device has the origin at the top-left corner, beginning at (0, 0). The x values increase to the right and y values increase going down. Each (x, y) coordinate defines the location of a single pixel.

The following example illustrates a few of the drawing functions and shows how to use the QPen and QBrush classes and how to set up the `paintEvent()` function for drawing on a widget.

```
# paint_basics.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QWidget
from PyQt5.QtGui import (QPainter, QPainterPath, QColor, QBrush, QPen, QFont, QPolygon, QLinearGradient
from PyQt5.QtCore import Qt, QPoint, QRect
class Drawing(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setFixedSize(600, 600)
        self.setWindowTitle('QPainter Basics')
        # Create a few pen colors
        self.black = '#000000'
        self.blue = '#2041F1'
        self.green = '#12A708'
        self.purple = '#6512F0'
        self.red = '#E00C0C'
        self.orange = '#FF930A'
        self.show()
    def paintEvent(self, event):
        """
        Create QPainter object and handle paint events.
        """
        painter = QPainter()
        painter.begin(self)
        # Use antialiasing to smooth curved edges
        painter.setRenderHint(QPainter.Antialiasing)
        self.drawPoints(painter)
        self.drawDiffLines(painter)
        self.drawText(painter)
        self.drawRectangles(painter)
        self.drawPolygons(painter)
        self.drawRoundedRects(painter)
        self.drawCurves(painter)
```

```
        self.drawCircles(painter)
        self.drawGradients(painter)
        painter.end()
    def drawPoints(self, painter):
        """
        Example of how to draw points with QPainter.
        """
        pen = QPen(QColor(self.black))
        for i in range(1, 9):
            pen.setWidth(i * 2)
            painter.setPen(pen)
            painter.drawPoint(i * 20, i * 20)
    def drawDiffLines(self, painter):
        """
        Examples of how to draw lines with QPainter.
        """
        pen = QPen(QColor(self.black), 2)
        painter.setPen(pen)
        painter.drawLine(230, 20, 230, 180)
        pen.setStyle(Qt.DashLine)
        painter.setPen(pen)
        painter.drawLine(260, 20, 260, 180)
        pen.setStyle(Qt.DotLine)
        painter.setPen(pen)
        painter.drawLine(290, 20, 290, 180)
        pen.setStyle(Qt.DashDotLine)
        painter.setPen(pen)
        painter.drawLine(320, 20, 320, 180)
        # Change the color and thickness of the pen
        blue_pen = QPen(QColor(self.blue), 4)
        painter.setPen(blue_pen)
        painter.drawLine(350, 20, 350, 180)
        blue_pen.setStyle(Qt.DashDotDotLine)
        painter.setPen(blue_pen)
        painter.drawLine(380, 20, 380, 180)
    def drawText(self, painter):
        """
        Example of how to draw text with QPainter.
        """
        text = "Don't look behind you."
        pen = QPen(QColor(self.red))
        painter.setFont(QFont("Helvetica", 15))
        painter.setPen(pen)
        painter.drawText(420, 110, text)
    def drawRectangles(self, painter):
        """
        Examples of how to draw rectangles with QPainter.
        """
        pen = QPen(QColor(self.black))
        brush = QBrush(QColor(self.black))
        painter.setPen(pen)
        painter.drawRect(20, 220, 80, 80)
        painter.setPen(pen)
        painter.setBrush(brush)
        painter.drawRect(120, 220, 80, 80)
        red_pen = QPen(QColor(self.red), 5)
        green_brush = QBrush(QColor(self.green))
        painter.setPen(red_pen)
        painter.setBrush(green_brush)
        painter.drawRect(20, 320, 80, 80)
        # Demonstrate how to change the alpha channel
```

```
        # to include transparency
        blue_pen = QPen(QColor(32, 85, 230, 100), 5)
        blue_pen.setStyle(Qt.DashLine)
        painter.setPen(blue_pen)
        painter.setBrush(green_brush)
        painter.drawRect(120, 320, 80, 80)
    def drawPolygons(self, painter):
        """
        Example of how to draw polygons with QPainter.
        """
        pen = QPen(QColor(self.blue), 2)
        brush = QBrush(QColor(self.orange))
        points = QPolygon([QPoint(240, 240), QPoint(380, 250),
                           QPoint(230, 380), QPoint(370, 360)])
        painter.setPen(pen)
        painter.setBrush(brush)
        painter.drawPolygon(points)
    def drawRoundedRects(self, painter):
        """
        Examples of how to draw rectangles with
        rounded corners with QPainter.
        """
        pen = QPen(QColor(self.black))
        brush = QBrush(QColor(self.black))
        rect_1 = QRect(420, 340, 40, 60)
        rect_2 = QRect(480, 300, 50, 40)
        rect_3 = QRect(540, 240, 40, 60)
        painter.setPen(pen)
        brush.setStyle(Qt.Dense1Pattern)
        painter.setBrush(brush)
        painter.drawRoundedRect(rect_1, 8, 8)
        brush.setStyle(Qt.Dense5Pattern)
        painter.setBrush(brush)
        painter.drawRoundedRect(rect_2, 5, 20)
        brush.setStyle(Qt.BDiagPattern)
        painter.setBrush(brush)
        painter.drawRoundedRect(rect_3, 15, 15)
    def drawCurves(self, painter):
        """
        Examples of how to draw curves with QPainterPath.
        """
        pen = QPen(Qt.black, 3)
        brush = QBrush(Qt.white)
        path = QPainterPath()
        path.moveTo(30, 420)
        path.cubicTo(30, 420, 65, 500, 30, 560)
        path.lineTo(163, 540)
        path.cubicTo(125, 360, 110, 440, 30, 420)
        path.closeSubpath()
        painter.setPen(pen)
        painter.setBrush(brush)
        painter.drawPath(path)
    def drawCircles(self, painter):
        """
        Example of how to draw ellipses with QPainter.
        """
        height, width = self.height(), self.width()
        center_x, center_y = (width / 2), height - 100
        radius_x, radius_y = 60, 60
        pen = QPen(Qt.black, 2, Qt.SolidLine)
        brush = QBrush(Qt.darkMagenta, Qt.Dense5Pattern)
```

```
            painter.setPen(pen)
            painter.setBrush(brush)
            painter.drawEllipse(QPoint(center_x, center_y), radius_x, radius_y)
        def drawGradients(self, painter):
            """
            Example of how to draw fill shapes using gradients.
            """
            pen = QPen(QColor(self.black), 2)
            gradient = QLinearGradient(450, 480, 520, 550)
            gradient.setColorAt(0.0, Qt.blue)
            gradient.setColorAt(0.5, Qt.yellow)
            gradient.setColorAt(1.0, Qt.cyan)
            painter.setPen(pen)
            painter.setBrush(QBrush(gradient))
            painter.drawRect(420, 420, 160, 160)
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = Drawing()
    sys.exit(app.exec_())
```

Listing 9-1 This code gives examples for drawing with the QPainter class

The results of different QPainter drawing functions can be seen in Figure 9-1.

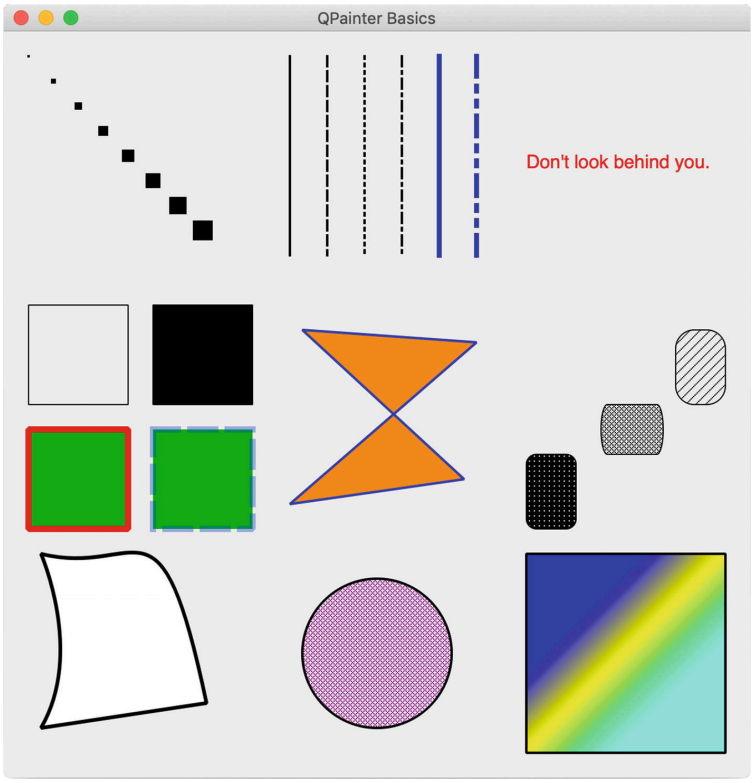

Figure 9-1 The window illustrates a few of the QPainter class's different functions. Starting from the top-left corner, the first row presents points, lines, and text; the second row illustrates shapes and patterns, including rectangles, polygons, and rectangles with rounded corners; the last row displays drawing curves, circles, and painting with gradients

Explanation

This program introduces quite a few new classes, a majority of them imported from the `QtGui` module. QtGui provides us with the tools we need for 2D graphics, imaging, and fonts. The `QPoint` and `QRect` classes imported from `QtCore` are used to define points and rectangles specified by coordinate values in the window's plane.

The `Drawing` class inherits from `QWidget`, and all drawing will occur on the widget's surface.

The paintEvent( ) Method

For general purposes, painting is handled inside the `paintEvent()` function . Let's take a look at how to set up QPainter in the following code to draw a simple line:

```
def paintEvent(self, event):
    painter = QPainter() # Construct the painter
    painter.begin(self)
    painter.drawLine(260, 20, 260, 180)
    painter.end()
```

Drawing occurs between the `begin()` and `end()` methods on the paint device, referenced by `self`. The drawing is handled in between these two methods. Using `begin()` and `end()` is not required. You could construct a painter that takes as a parameter the paint device. However, `begin()` and `end()` can be used to catch any errors should the painter fail.

Other methods can also be called during the paint event. Since only one painter is allowed at a time, in the preceding example, we call different methods that all take the painter object as an argument.

```
self.drawPoints(painter)
self.drawLines(painter)
```

One of the settings that we can change in QPainter is the rendering quality using render hints. `QPainter.Antialiasing` creates smoother-looking curved edges.

```
painter.setRenderHint(QPainter.Antialiasing)
```

The QColor, QPen, and QBrush Classes

Some of the settings that can be modified include the color, width, and styles used to draw lines and shapes. The **QColor** class provides access to different color schemes, for example, RGB, HSV, and CMYK values. Colors can be specified by using either RGB hexadecimal strings, `'#112233'`; predefined color names, `Qt.blue` or `Qt.darkBlue`; or RGB values, `(233, 12, 43)`. QColor also includes an alpha channel used for giving colors transparency, where 0 is completely transparent and 255 is completely opaque. Listing **9-1** demonstrates all three of these types.

QPen is used for drawing lines and the outlines of shapes. The following code creates a black pen with a width of 2 pixels that draws dashed lines. The default style is `Qt.SolidLine`.

```
pen = QPen(QColor('#000000'), 2, Qt.DashLine)
painter.setPen(pen)
```

QBrush defines how to paint, that is, fill in, shapes. Brushes can have a color, a pattern, a gradient, or a texture. A magenta brush with the `Dense5Pattern` style is created with the following code. The default style is `Qt.SolidPattern`.

```
brush = QBrush(Qt.darkMagenta, Qt.Dense5Pattern)
painter.setBrush(brush)
```

If you wish to create multiple lines or shapes with different pens and brushes, make sure to call `setPen()` and/or `setBrush()` each time they need to be changed. Otherwise, QPainter will continue to use the pen and brush settings from the previous call.

> **Note** Calling `QPainter.begin()` will reset all the painter settings to default values.

Drawing Points and Lines

The `drawPoint()` method can be used to draw single pixels. By changing the width of the pen, you can draw wider points. The x and y values can either be explicitly defined or specified by using QPoint. An example of points is shown in Figure 9-2.

```
pen.setWidth(3)
painter.setPen(pen)
    painter.drawPoint(10, 15)
```

> **Note** The `drawPoint()` and other methods are specified using integer values. Some of the drawing methods allow you to also use floating-point values. Rather than import the `QPoint` and `QRect` classes, you should use `QPointF` and `QRectF`.

For drawing lines, there are the `drawLine()` or `drawLines()` methods. Each of the lines shown in Figure 9-2 displays different styles, widths, or colors. Lines are created by specifying a set of points, the starting `x1` and `y1` values and the ending `x2` and `y2` values.

```
pen.setStyle(Qt.DashLine) # Specify a style
painter.setPen(pen) # Set the pen
painter.drawLine(260, 20, 260, 180) # x1, y1, x2, y2
```

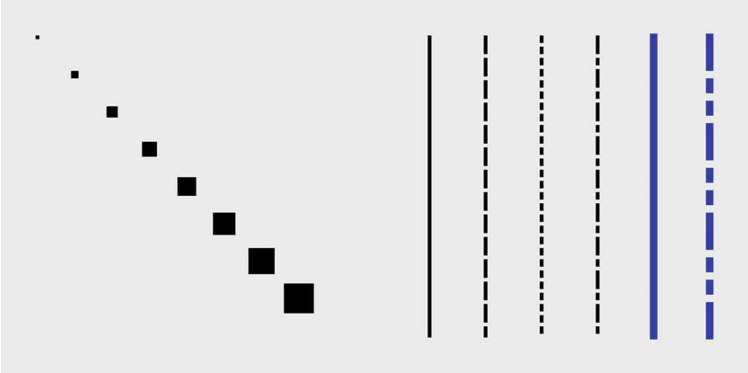

Figure 9-2 Example of points and lines drawn using QPainter

Drawing Text

The `drawText()` method is used to draw text on the paint device. An example of drawing text can be seen in Figure 9-3. We can make use of `setFont()` to apply different font settings.

```
painter.setFont(QFont("Helvetica", 15))
painter.setPen(pen)
painter.drawText(420, 110, text)
```

The `text` is drawn by first specifying the top-left coordinates on the paint device (think of text as being placed inside of a rectangle). This is the simplest way to draw text. For multiple lines or for wrapping text, use a QRect rectangle to contain the text.

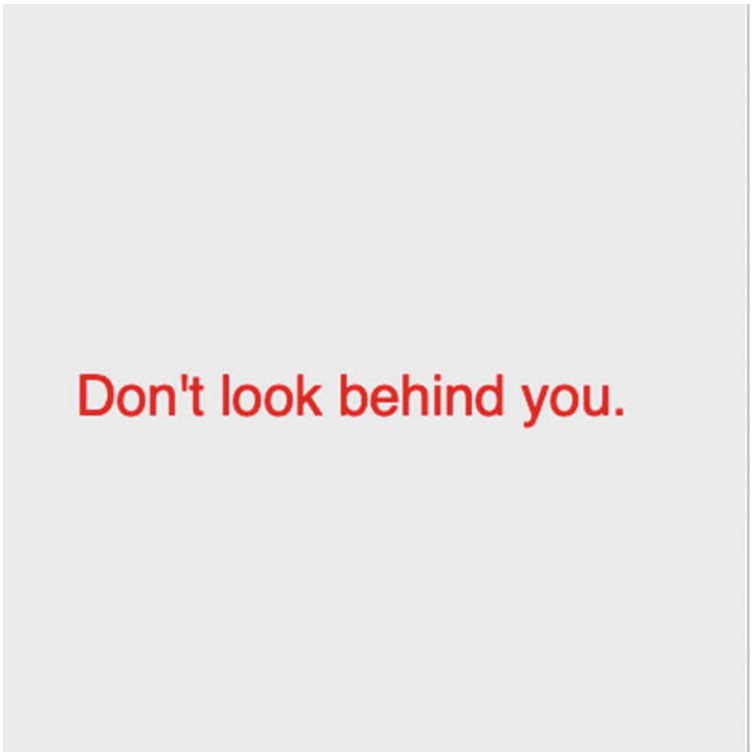

Figure 9-3 A simple example of drawing text with QPainter

Drawing Two-Dimensional Shapes

There are a few different ways to draw quadrilaterals using the `drawRect()` method . For this example, we will specify the top-left corner's coordinates followed by the width and height of the shape.

```
painter.drawRect(120, 220, 80, 80)
```

For each of the squares shown in the top-left corner of Figure 9-4, we begin by setting the pen and brush values before calling `drawRect()` to draw the shape. The first shape has a black pen with no brush; the second calls `setBrush()` to fill in the square. The next shape uses a red pen with a green brush. Finally, the last square shows an example of how to set the transparency of the pen object's color to `100`.

```
blue_pen = QPen(QColor(32, 85, 230, 100), 5)
```

To draw irregular polygons, the `QPolygon` class can be used by specifying the point coordinates of each corner. The order that the points are created in the QPolygon object is the order in which they are drawn. The polygon object is then drawn using `drawPolygon()`. The polygon can be seen in the middle of the top row in Figure 9-4.

```
painter.drawPolygon(points)
```

QPainter can also draw rectangles with rounded corners. The process for drawing them is similar to drawing normal rectangles, except we need to specify the x and y radius values for the corners. Examples can be seen in Figure 9-4 in the top-right corner. The following snippet of code shows how to create a rounded rectangle by first creating the `QRect` rectangle and then specifying the style:

```
rect_1 = QRect(420, 340, 40, 60) # x, y, width, height
brush.setStyle(Qt.Dense1Pattern)
painter.setBrush(brush)
painter.drawRoundedRect(rect_1, 8, 8) # rect, x_rad, y_rad
```

For drawing abstract shapes, we need to use **QPainterPath**. Objects composed of different components, such as lines, rectangles, or curves, are called **painter paths**. An example of a painter path can be seen in the bottom-left corner of Figure 9-4.

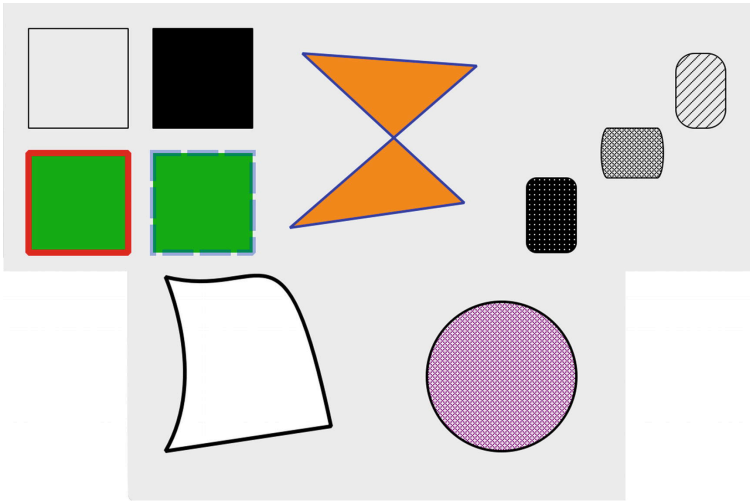

Figure 9-4 Different shapes drawn with QPainter

In the `drawCurves()` method of the earlier program, we first create a black `pen` and a white `brush` and an instance of `QPainterPath`. The `moveTo()` method moves to a position in the window without drawing any other components. We will start drawing at this position, `(30, 420)`.

```
path.cubicTo(30, 420, 65, 500, 30, 560)
path.lineTo(163, 540)
path.cubicTo(125, 360, 110, 440, 30, 420)
```

The `cubicTo()` method can be used to draw a parametric curve, also called a Bézier curve, from the starting position we moved to and the ending position, `(30, 560)`. The first two points, `(30, 420)` and `(65, 500)`, in `cubicTo()` are used to influence how the line curves between the start-

ing and ending points. The next components of `path` are a line drawn with `lineTo()` and another curve. The abstract shape is closed with `closeSubpath()`, and the path is drawn using `drawPath()`.

The last shape we are going to look at is the ellipse which is drawn using `drawEllipse()`. For an ellipse, we need four values, the location of the center, and two radii values for the x and y directions. If the radii values are equal, we can draw a circle, like in the bottom-right corner of Figure 9-4. The following code shows how to draw an ellipse with a QPoint as the center coordinate, but the shape can also be drawn by defining a QRect.

```
painter.drawEllipse(QPoint(center_x, center_y), radius_x, radius_y)
```

Drawing Gradients

Gradients can be used along with QBrush to fill the inside of shapes. There are three different types of gradient styles in PyQt – linear, radial, and conical. For this example, we will use the **QLinearGradient** class to interpolate colors between two start and end points. The result can be seen in Figure 9-5.

The QLinearGradient constructor takes as arguments the area of the paint device where the gradient will occur, specified by the `x1`, `y1`, `x2`, `y2` coordinates.

```
gradient = QLinearGradient(450, 480, 520, 550)
```

We can create points to start and stop painting colors using `setStopPoint()` . This method defines where one color ends and another color begins.

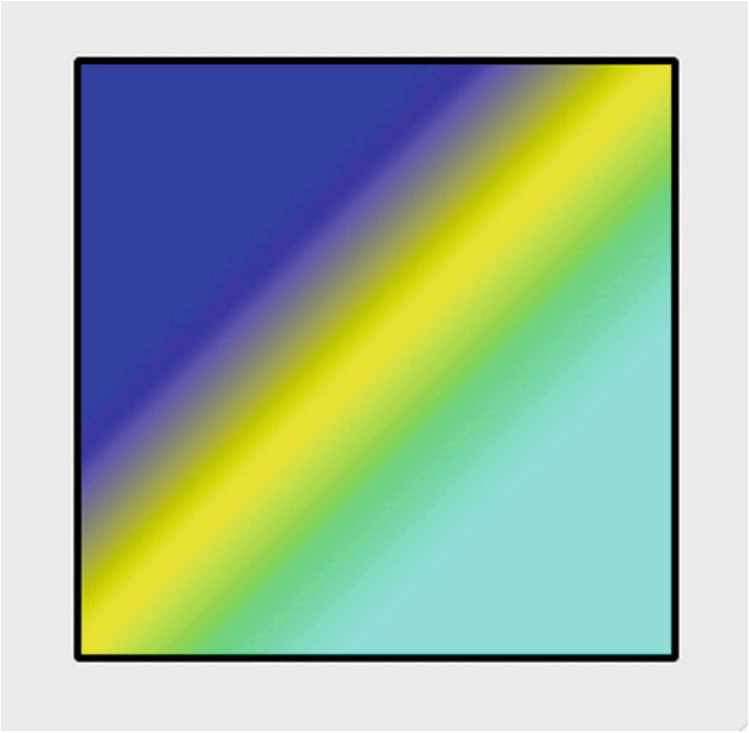

Figure 9-5 Applying a gradient to a square

Project 9.1 – Painter GUI

There are many digital art applications out there, filled to the brim with tools for drawing, painting, editing, and creating your own art on the computer. With QPainter, you could manually code each individual line and shape one by one. However, rather than going through that painstaking process to create digital works of art, the painter GUI project lays the foundation for creating your drawing application that could pave the way for a smoother drawing process. The interface can be seen in Figure 9-6.

For this first project, we will be looking to combine many of the concepts that you learned in previous chapters, including menubars, toolbars, status bars, dialog boxes, creating icons, and reimplementing event handlers, and combine them with the QPainter class. On top of it all, we will be sprinkling on a few new ideas, focusing on how to create tool tips and track the mouse's position.

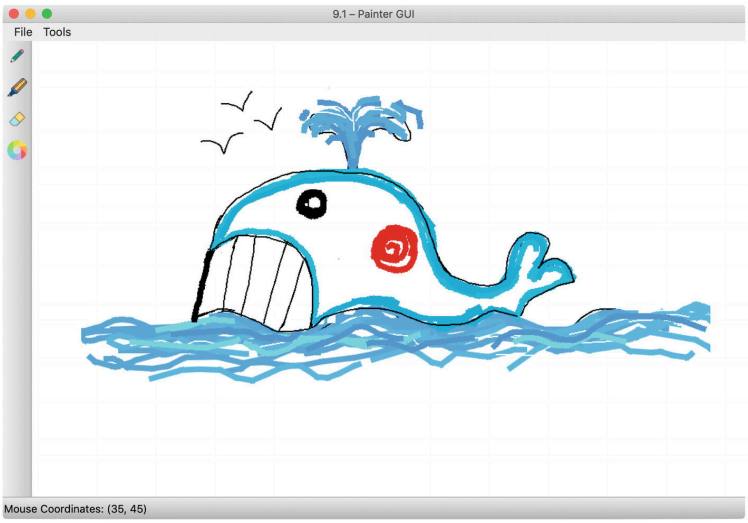

Figure 9-6 The painter GUI with toolbar on the left side of the window and the mouse's current coordinates displayed in the status bar

Painter GUI Solution

For the painter GUI in Listing 9-2, users will be able to draw using either a pencil or a marker tool, erase, and select colors using the QColorDialog. The items in the menu allow users to clear the current canvas, save their drawing, quit, and turn on or off antialiasing.

```
# painter.py
# Import necessary modules
import os, sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QAction, QLabel, QToolBar, QStatusBar, QToolTip
from PyQt5.QtGui import (QPainter, QPixmap, QPen, QColor, QIcon, QFont)
from PyQt5.QtCore import Qt, QSize, QPoint, QRect
# Create widget to be drawn on
class Canvas(QLabel):
    def __init__(self, parent):
        super().__init__(parent)
        width, height = 900, 600
        self.parent = parent
        self.parent.setFixedSize(width, height)
        # Create pixmap object that will act as the canvas
        pixmap = QPixmap(width, height) # width, height
```

```
        pixmap.fill(Qt.white)
        self.setPixmap(pixmap)
        # Keep track of the mouse for getting mouse coordinates
        self.mouse_track_label = QLabel()
        self.setMouseTracking(True)
        # Initialize variables
        self.antialiasing_status = False
        self.eraser_selected = False
        self.last_mouse_pos = QPoint()
        self.drawing = False
        self.pen_color = Qt.black
        self.pen_width = 2
    def selectDrawingTool(self, tool):
        """
        Determine which tool in the toolbar has been selected.
        """
        if tool == "pencil":
            self.eraser_selected = False
            self.pen_width = 2
        elif tool == "marker":
            self.eraser_selected = False
            self.pen_width = 8
        elif tool == "eraser":
            self.eraser_selected = True
        elif tool == "color":
            self.eraser_selected = False
            color = QColorDialog.getColor()
            if color.isValid():
                self.pen_color = color
    def mouseMoveEvent(self, event):
        """
        Handle mouse movements.
        Track coordinates of mouse in window and display in the status bar.
        """
        mouse_pos = event.pos()
        if (event.buttons() and Qt.LeftButton) and self.drawing:
            self.mouse_pos = event.pos()
            self.drawOnCanvas(mouse_pos)
        self.mouse_track_label.setVisible(True)
        sb_text = "Mouse Coordinates: ({}, {})".format(mouse_pos.x(), mouse_pos.y())
        self.mouse_track_label.setText(sb_text)
        self.parent.status_bar.addWidget(self.mouse_track_label)
    def drawOnCanvas(self, points):
        """
        Performs drawing on canvas.
        """
        painter = QPainter(self.pixmap())
        if self.antialiasing_status:
            painter.setRenderHint(QPainter.Antialiasing)
        if self.eraser_selected == False:
            pen = QPen(QColor(self.pen_color), self.pen_width)
            painter.setPen(pen)
            painter.drawLine(self.last_mouse_pos, points)
            # Update the mouse's position for next movement
            self.last_mouse_pos = points
        elif self.eraser_selected == True:
            # Use the eraser
            eraser = QRect(points.x(), points.y(), 12, 12)
            painter.eraseRect(eraser)
        painter.end()
        self.update()
```

```
    def newCanvas(self):
        """
        Clears the current canvas.
        """
        self.pixmap().fill(Qt.white)
        self.update()
    def saveFile(self):
        """
        Save a .png image file of current pixmap area.
        """
        file_format = "png"
        default_name = os.path.curdir + "/untitled." + file_format
        file_name, _ = QFileDialog.getSaveFileName(self, "Save As",
                       default_name, "PNG Format (*.png)")
        if file_name:
            self.pixmap().save(file_name, file_format)
    def mousePressEvent(self, event):
        """
        Handle when mouse is pressed.
        """
        if event.button() == Qt.LeftButton:
            self.last_mouse_pos = event.pos()
            self.drawing = True
    def mouseReleaseEvent(self, event):
        """
        Handle when mouse is released.
        Check when eraser is no longer being used.
        """
        if event.button() == Qt.LeftButton:
            self.drawing = False
        elif self.eraser_selected == True:
            self.eraser_selected = False
    def paintEvent(self, event):
        """
        Create QPainter object.
        This is to prevent the chance of the painting being lost
        if the user changes windows.
        """
        painter = QPainter(self)
        target_rect = QRect()
        target_rect = event.rect()
        painter.drawPixmap(target_rect, self.pixmap(), target_rect)
class PainterWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        #self.setMinimumSize(700, 600)
        self.setWindowTitle('9.1 – Painter GUI')
        QToolTip.setFont(QFont('Helvetica', 12))
        self.createCanvas()
        self.createMenu()
        self.createToolbar()
        self.show()
    def createCanvas(self):
        """
        Create the canvas object that inherits from QLabel.
        """
```

```
        self.canvas = Canvas(self)
        # Set the main window's central widget
        self.setCentralWidget(self.canvas)
    def createMenu(self):
        """
        Set up the menu bar and status bar.
        """
        # Create file menu actions
        new_act = QAction('New Canvas', self)
        new_act.setShortcut('Ctrl+N')
        new_act.triggered.connect(self.canvas.newCanvas)
        save_file_act = QAction('Save File', self)
        save_file_act.setShortcut('Ctrl+S')
        save_file_act.triggered.connect(self.canvas.saveFile)
        quit_act = QAction("Quit", self)
        quit_act.setShortcut('Ctrl+Q')
        quit_act.triggered.connect(self.close)
        # Create tool menu actions
        anti_al_act = QAction('AntiAliasing', self, checkable=True)
        anti_al_act.triggered.connect(self.turnAntialiasingOn)
        # Create the menu bar
        menu_bar = self.menuBar()
        menu_bar.setNativeMenuBar(False)
        # Create file menu and add actions
        file_menu = menu_bar.addMenu('File')
        file_menu.addAction(new_act)
        file_menu.addAction(save_file_act)
        file_menu.addSeparator()
        file_menu.addAction(quit_act)
        # Create tools menu and add actions
        file_menu = menu_bar.addMenu('Tools')
        file_menu.addAction(anti_al_act)
        self.status_bar = QStatusBar()
        self.setStatusBar(self.status_bar)
    def createToolbar(self):
        """
        Create toolbar to contain painting tools.
        """
        tool_bar = QToolBar("Painting Toolbar")
        tool_bar.setIconSize(QSize(24, 24))
        # Set orientation of toolbar to the left side
        self.addToolBar(Qt.LeftToolBarArea, tool_bar)
        tool_bar.setMovable(False)
        # Create actions and tool tips and add them to the toolbar
        pencil_act = QAction(QIcon("icons/pencil.png"), 'Pencil', tool_bar)
        pencil_act.setToolTip('This is the <b>Pencil</b>.')
        pencil_act.triggered.connect(lambda: self.canvas.selectDrawingTool("pencil"))
        marker_act = QAction(QIcon("icons/marker.png"), 'Marker', tool_bar)
        marker_act.setToolTip('This is the <b>Marker</b>.')
        marker_act.triggered.connect(lambda: self.canvas.selectDrawingTool("marker"))
        eraser_act = QAction(QIcon("icons/eraser.png"), "Eraser", tool_bar)
        eraser_act.setToolTip('Use the <b>Eraser</b> to make it all disappear.')
        eraser_act.triggered.connect(lambda: self.canvas.selectDrawingTool("eraser"))
        color_act = QAction(QIcon("icons/colors.png"), "Colors", tool_bar)
        color_act.setToolTip('Choose a <b>Color</b> from the Color dialog.')
        color_act.triggered.connect(lambda: self.canvas.selectDrawingTool("color"))
        tool_bar.addAction(pencil_act)
        tool_bar.addAction(marker_act)
        tool_bar.addAction(eraser_act)
        tool_bar.addAction(color_act)
    def turnAntialiasingOn(self, state):
```

```
            """
            Turn antialiasing on or off.
            """
            if state:
                self.canvas.antialiasing_status = True
            else:
                self.canvas.antialiasing_status = False
        def leaveEvent(self, event):
            """
            QEvent class that is called when mouse leaves screen's space. Hide mouse coordinates in status
            the window.
            """
            self.canvas.mouse_track_label.setVisible(False)
if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setAttribute(Qt.AA_DontShowIconsInMenus, True)
    window = PainterWindow()
    sys.exit(app.exec_())
```

Listing 9-2 The code for creating the painter GUI

The final GUI can be seen in Figure 9-6.

Explanation

The painter GUI allows users to draw images on the canvas area. Unlike in the previous example where painting occurred on the main widget, for this example we will see how to subclass QLabel and reimplement its painting and mouse event handlers. The handling for some of the event handlers in this application was adapted from the Qt document web site.[1]

The program contains two classes – the `Canvas` class for drawing and the `PainterWindow` class for creating the menu and toolbar.

Creating the Canvas Class

Subclassing `QLabel` and reimplementing its `paintEvent()` method is a much easier way to manage drawing on a label object. We then create a `pixmap` and pass it to `setPixmap()`. Since QPixmap can be used as a QPaintDevice, using a pixmap makes handling the drawing and displaying of pixels much simpler. Also, using QPixmap means that we can set an initial background color using `fill()`.

Next, we need to initialize a few variables and objects.

- `mouse_track_label` – A label for displaying the mouse's current position
- `eraser_selected` – `True` if the eraser is selected
- `antialiasing_status` – `True` if the user has checked the menu item for using antialiasing
- `last_mouse_pos` – Keep track of the mouse's last position when the left mouse button is pressed or when the mouse moves
- `drawing` – `True` if the left mouse button is pressed, indicating the user might be drawing
- `pen_color`, `pen_width` – Variables that hold the initial values of the pen and brush

Since the user will use the mouse to draw in the GUI window, we need to handle the events when the mouse button is pressed or released and when the mouse is moved. We can use `setMouseTracking()` to keep track of the mouse cursor and return its coordinates in `mouseMoveEvent()`. Those coordinates are displayed in the status bar.

If the user presses the left mouse button while the cursor is in the window, we set `drawing` equal to `True` and store the current value of the mouse in `last_mouse_pos`. Then `drawOnCanvas()` is called in the `mouseMoveEvent()`.

The user has four choices in the toolbar, including a pencil, marker, eraser, and a color selector. If a user selects a tool from the toolbar, a signal triggers the `selectDrawingTool()` slot, updating the current tool and painter settings.

The actual drawing is handled in `drawOnCanvas()`. An instance of `QPainter` is created that draws on the `pixmap`. We also check whether the `eraser_selected` is `True` or `False` to test whether we can draw or erase. The reimplementation of the `paintEvent()` creates a painter for the canvas area and draws the pixmap using `drawPixmap()`. By first drawing on a `QPixmap` in the `drawOnCanvas()` method and then copying the `QPixmap` onto the screen in the `paintEvent()`, we can ensure that our drawing won't be lost if the window is minimized.

The `Canvas` class also includes methods for clearing and saving the pixmap.

Creating the PainterWindow Class

The `PainterWindow` class creates the main menu, toolbar, tool tips for each of the buttons in the toolbar, and an instance of the `Canvas` class.

The File menu contains actions for clearing the canvas, saving the image, and quitting the application. The Tools menu contains a checkable menu item that turns antialiasing on or off.

The toolbar creates the actions and icons for the drawing tools. If a button is clicked, it triggers the `Canvas` class's `selectDrawingTool()` slot.

The reimplemented `leaveEvent()` handles if the mouse cursor moves outside the main window and sets the `mouse_track_label`'s visibility to `False`.

Handling Mouse Movement Events

This project displays the mouse's current x and y coordinates in the status bar. You may not want this kind of functionality, so the following code shows the basics for turning mouse tracking on and setting up `mouseMoveEvent()` to return the x and y values:

```
# Turn mouse tracking on
self.setMouseTracking(True)
```

```
    def mouseMoveEvent(self, event):
        mouse_pos = event.pos()
        pos_text = "Mouse Coordinates: ({}, {})".format(mouse_pos.x(), mouse_pos.y())
        print(pos_text)
```

Mouse move events occur whenever the mouse is moved, or when a mouse button is pressed or released.

Creating Tool Tips for Widgets

A user may often find themselves wondering about what some widget or action in a menu or toolbar actually does in an application. Tool tips are useful little bits of text that can be displayed to inform someone of a widget's function. Tool tips can be applied to any widget by using the `setToolTip()` method . Tips can display rich text formatted strings as shown in the following sample of code and in Figure 9-7. The font style and appearance of a tool tip can be adapted to fit your preferences.

```
eraser_act.setToolTip('Use the <b>Eraser</b> to make it all disappear.')
```

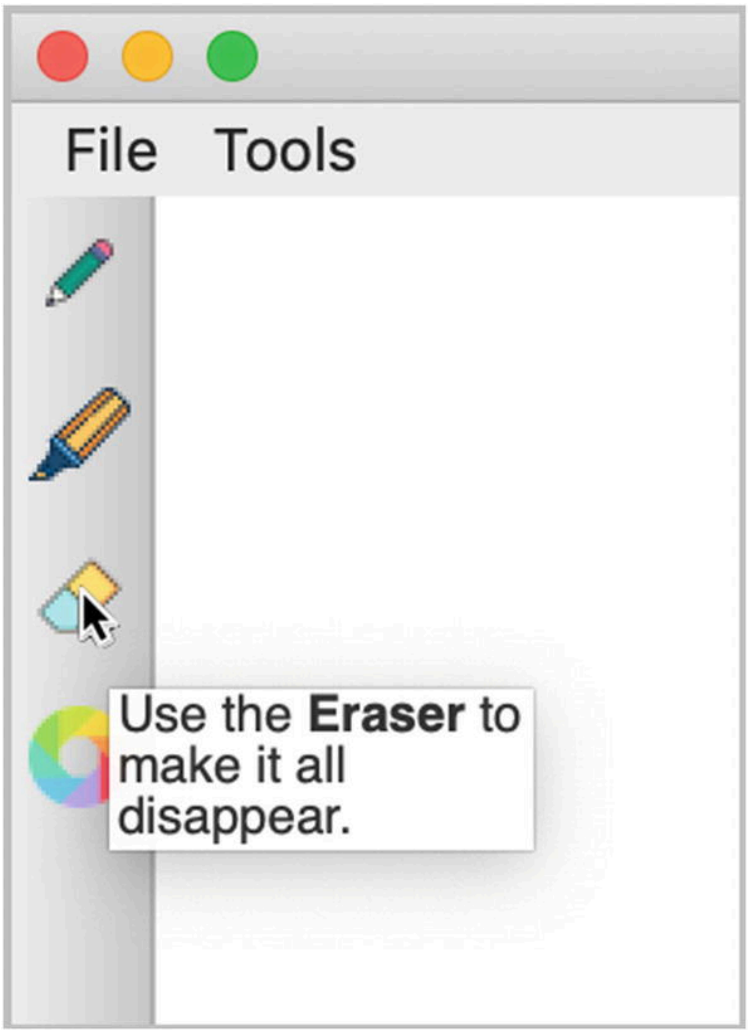

Figure 9-7 The tool tip that is displayed when the user hovers over the eraser button

Project 9.2 – Animation with QPropertyAnimation

The following project serves as an introduction to Qt's Graphics View Framework and the QAnimationProperty class. With the framework, ap-

plications can be created that allow users to interact with the items in the window.

A Graphics View is comprised of three components:

1. A **scene** created from the `QGraphicsScene` class. The scene creates the surface for managing 2D graphical items and must be created along with a view to visualize a scene.
2. `QGraphicsView` provides the **view** widget for visualizing the elements of a scene, creating a scroll area that allows user to navigate in the scene.
3. **Items** in the scene are based on the `QGraphicsItem` class. Users can interact with graphical items through mouse and key events, and drag and drop. Items also support collision detection.

QAnimationProperty is used to animate the properties of widgets and items. Animations in GUIs can be used for animating widgets. For example, you could animate a button that grows, shrinks, or rotates, or text that smoothly moves around in the window, or create widgets that fade in and out or change colors. QAnimationProperty only works with objects that inherit the **QObject** class. QObject is the base class for all objects created in PyQt.

Qt provides a number of simple items that inherit QGraphicsItem, including basic shapes, text, and pixmaps. These items already provide support for mouse and keyboard interaction. However, QGraphicsItem does not inherit QObject. Therefore, if you want to animate a graphics item with QPropertyAnimation, you must first create a new class that inherits from QObject and define new properties for the item.

Figure 9-8 shows an example of the scene we are going to create in this project.

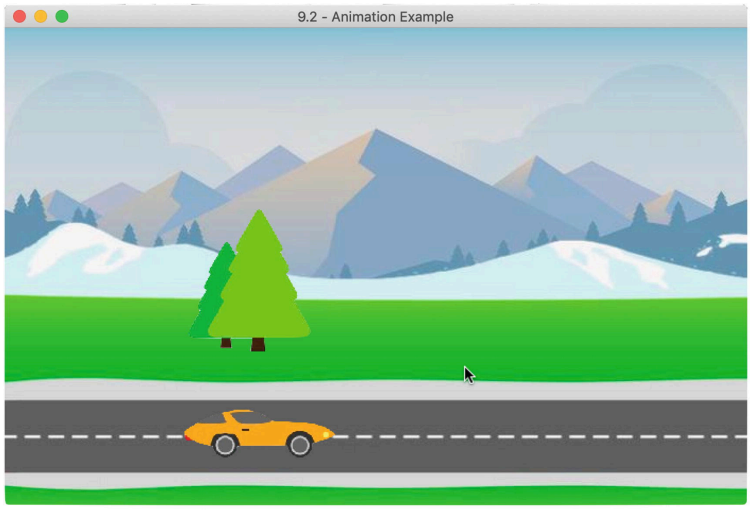

Figure 9-8 The car and tree objects that move across the scene in the window

Animation Solution

In the following application, you will find out how to create new properties for items using pyqtProperty, learn how to animate objects using the

QPropertyAnimation class, and create a Qt Graphics View for displaying the items and animations. The code for creating simple animations can be found in Listing 9-3.

```
# animation.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QGraphicsView, QGraphicsScene, QGraphicsPixmapItem)
from PyQt5.QtCore import (QObject, QPointF, QRectF,
    QPropertyAnimation, pyqtProperty)
from PyQt5.QtGui import QPixmap
# Create Objects class that defines the position property of
# instances of the class using pyqtProperty.
class Objects(QObject):
    def __init__(self, image_path):
        super().__init__()
        item_pixmap = QPixmap(image_path)
        resize_item = item_pixmap.scaledToWidth(150)
        self.item = QGraphicsPixmapItem(resize_item)
    def _set_position(self, position):
        self.item.setPos(position)
    position = pyqtProperty(QPointF, fset=_set_position)
class AnimationScene(QGraphicsView):
    def __init__(self):
        super().__init__()
        self.initializeView()
    def initializeView(self):
        """
        Initialize the graphics view and display its contents
        to the screen.
        """
        self.setGeometry(100, 100, 700, 450)
        self.setWindowTitle('9.2 - Animation Example')
        self.createObjects()
        self.createScene()
        self.show()
    def createObjects(self):
        """
        Create instances of the Objects class, and set up the objects
        animations.
        """
        # List that holds all of the animations.
        animations = []
        # Create the car object and car animation.
        self.car = Objects('images/car.png')
        self.car_anim = QPropertyAnimation(self.car, b"position")
        self.car_anim.setDuration(6000)
        self.car_anim.setStartValue(QPointF(-50, 350))
        self.car_anim.setKeyValueAt(0.3, QPointF(150, 350))
        self.car_anim.setKeyValueAt(0.6, QPointF(170, 350))
        self.car_anim.setEndValue(QPointF(750, 350))
        # Create the tree object and tree animation.
        self.tree = Objects('images/trees.png')
        self.tree_anim = QPropertyAnimation(self.tree, b"position")
        self.tree_anim.setDuration(6000)
        self.tree_anim.setStartValue(QPointF(750, 150))
        self.tree_anim.setKeyValueAt(0.3, QPointF(170, 150))
        self.tree_anim.setKeyValueAt(0.6, QPointF(150, 150))
        self.tree_anim.setEndValue(QPointF(-150, 150))
        # Add animations to the animations list, and start the
```

```
            # animations once the program begins running.
            animations.append(self.car_anim)
            animations.append(self.tree_anim)
            for anim in animations:
                anim.start()
        def createScene(self):
            """
            Create the graphics scene and add Objects instances
            to the scene.
            """
            self.scene = QGraphicsScene(self)
            self.scene.setSceneRect(0, 0, 700, 450)
            self.scene.addItem(self.car.item)
            self.scene.addItem(self.tree.item)
            self.setScene(self.scene)
        def drawBackground(self, painter, rect):
            """
            Reimplement QGraphicsView's drawBackground() method.
            """
            scene_rect = self.scene.sceneRect()
            background = QPixmap("images/highway.jpg")
            bg_rectf = QRectF(background.rect())
            painter.drawPixmap(scene_rect, background, bg_rectf)
    if __name__ == '__main__':
        app = QApplication(sys.argv)
        window = AnimationScene()
        sys.exit(app.exec_())
```

Listing 9-3 Code for animating objects in Qt's Graphics View Framework

A still image from the animation project is shown in Figure 9-8.

Explanation

Since we are going to create a Graphics Scene, we need to import `QGraphicsScene`, `QGraphicsView`, and one of the `QGraphicsItem` classes. For this program, we import `QGraphicsPixmapItem` since we will be working with pixmaps. New Qt properties can be made using `pyqtProperty`.

Since QObject does not have a position property, we need to define one with pyqtProperty in the `Objects` class. `QGraphicsPixmapItem()` creates a pixmap that can be added into the `QGraphicsScene`. We create a `position` property that allows us to set and update the position of the object using `fset`. `_set_position()` passes the `position` to the `QGraphicsItem.setPos()` method , setting the position of the item to the coordinates specified by `QPointF`. Underscores in front of variable, method, or class names are used to denote private functions.

```
    def _set_position(self, position):
        self.item.setPos(position)
    position = pyqtProperty(QPointF, fset=_set_position)
```

The goal of this project is to animate two items, a car and a tree, in a QGraphicsScene. Let's first create the objects and the animations that will be placed into the scene. For this scene, the two items will move at the same time. Qt provides other classes for handling groups of animations, but for this example, the `QPropertyAnimation` and the `animations` list are used to keep track of the multiple animations.

Create the `car` item as an instance of the `Objects` class, and pass `car` and the `position` setter to `QPropertyAnimation()`. QPropertyAnimation will update the `position`'s value so that the `car` moves across the scene. To animate items, use `setDuration()` to set the amount of time the object moves in milliseconds, and specify start and end values of the property with `setStartValue()` and `setEndValue()`. The animation for the `car` is 6 seconds and starts off-screen on the left side and travels to the right. The tree is set up in a similar manner, but traveling in the opposite direction.

The `setKeyValueAt()` method allows us to create key frames at the given steps with the specified `QPointF` values. Using the key frames, the car and tree will appear to slow down as they pass in the scene. The `start()` method begins the animation.

Setting up a scene is simple. Create an instance of the `scene`, set the scene's size, add objects and their animations using `addItem()`, and then call `setScene()`.

Finally, a scene can be given a background using QBrush. If you want to use a background image, you will need to reimplement the QGraphicView's `drawBackground()` method like we do in this example.

Project 9.3 – RGB Slider Custom Widget

For this chapter's final project, we are going to take a look at making a custom, functional widget in PyQt. While PyQt offers a variety of widgets for building GUIs, every once in a while you might find yourself needing to create your own. One of the benefits of creating a customized widget is that you can either design a general widget that can be used by many different applications or create an application-specific widget that allows you to solve a specific problem.

There are quite a few techniques that you can use to create your own widgets, most which we have already seen in previous examples.

- Modifying the properties of PyQt's widgets by using built-in methods, such as `setAlignment()`, `setTextColor()`, and `setRange()`
- Creating style sheets to change a widget's existing behavior and appearances
- Subclassing widgets and reimplementing event handlers, or adding properties dynamically to QObject classes
- Creating composite widgets which are made up of two more types of widgets and arranged together using a layout
- Designing a completely new widget that subclasses QWidget and has its own unique properties and appearance

The RGB slider, shown in Figure 9-9, actually is created by combining a few of the preceding techniques listed. The widget uses Qt's QSlider and QSpinBox widgets for selecting RGB values and displays the color on a QLabel. The look of the sliders is modified by using style sheets. All of the

widgets are then assembled into a parent widget which we can then import into other PyQt applications.

PyQt's Image Handling Classes

In previous examples, we have only worked with QPixmap for handling image data. Qt actually provides four different classes for working with images, each with their own special purposes.

QPixmap is the go-to choice for displaying images on the screen. Pixmaps can be used on QLabel widgets, or even on push buttons and other widgets that can display icons. **QImage** is optimized for reading, writing, and manipulating images, allowing direct access to an image's pixel data. QImage can also act as paint device.

Conversion between QImage and QPixmap is also possible. One possibility for using the two classes together is to load an image file with QImage, manipulate the image data, and then convert the image to a pixmap before displaying it on the screen. The RGB slider widget shows an example of how to convert between the two classes.

QBitmap is a subclass of QPixmap and provides monochrome (1-bit depth) pixmaps. **QPicture** is a paint device that replays QPainter commands, that is, you can create a picture from whatever device you are painting on. Pictures created with QPicture are resolution independent, appearing the same on any device.

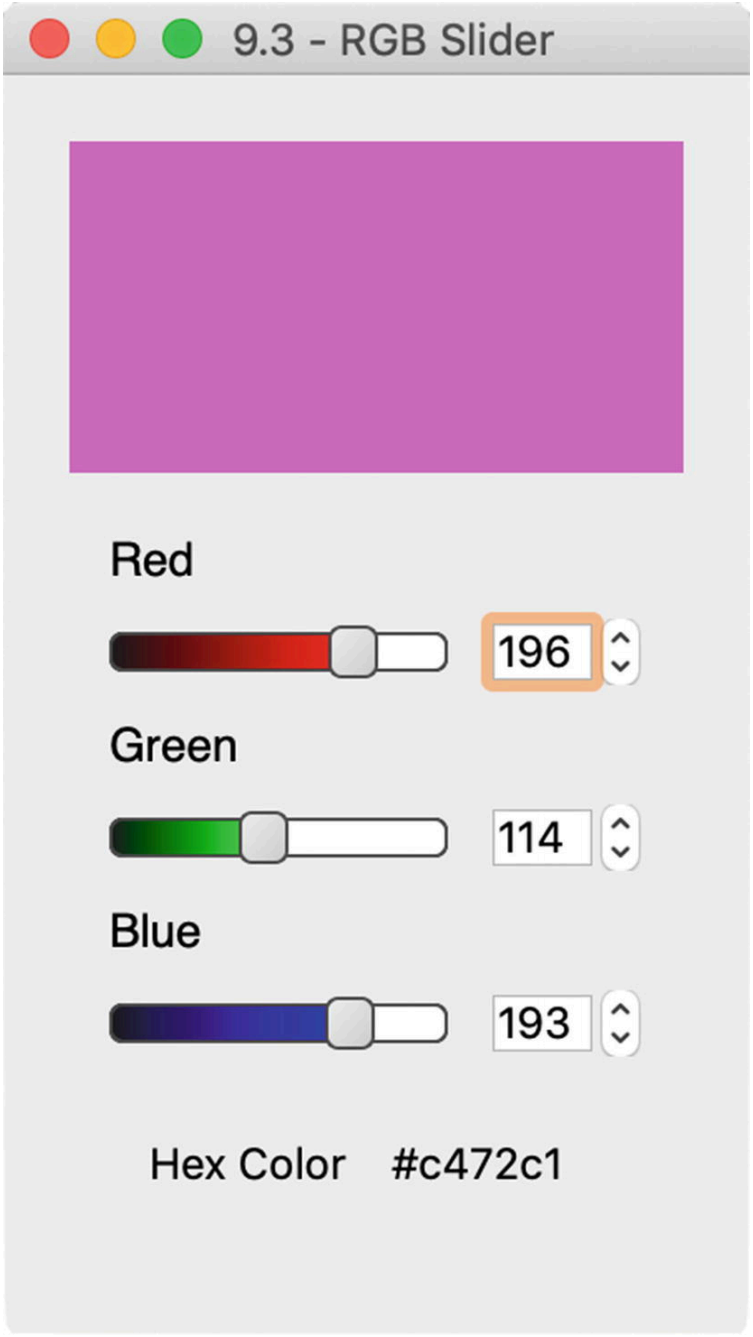

Figure 9-9 A custom widget used to select colors using sliders and spin boxes

Our custom widget uses two types of widgets for selecting RGB values – QSpinBox, which was introduced in Chapter 4, and a new widget, QSlider.

The QSlider Widget

The **QSlider** class provides a developer with a tool for selecting integer values within a bounded range. Sliders provide users with a convenient means for quickly selecting values or changing settings with only the movement of a simple handle. By default, sliders are arranged vertically, but that can be changed by passing `Qt.Horizontal` to the constructor.

In the following bit of code, you can see how to create an instance of QSlider, set the slider's maximum range value, and connect to `valueChanged()` to emit a signal when the slider's value has changed:

```
slider = QSlider(Qt.Horizontal, self)
# Default values are from 0 to 99
slider.setMaximum(200)
slider.valueChanged[int].connect(self.printSliderValue)
def printSliderValue(self, value):
    print(value)
```

An example of stylized slider widgets can be seen in Figure 9-9.

RGB Slider Solution

The RGB slider, which can be found in Listing 9-4, is a custom widget created by combining a few of Qt's built-in widgets – QLabel, QSlider, and QSpinBox. The appearance of the sliders is adjusted using style sheets so that they give visual feedback to the user about which RGB value they are adjusting. The sliders and spin boxes are connected together so that their values are in sync and so that the user can see the integer value on the RGB scale. The RGB values are also converted to hexadecimal format and displayed on the widget.

The sliders and spin boxes can be used to either find out the RGB or hexadecimal values for a color or use the reimplemented `mousePressEvent()` method so that a user can click a pixel in an image to find out its value. An example of this is shown in Listing 9-5, where you will also see how to import the RGB slider in a demo application.

```
# rgb_slider.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel,
    QSlider, QSpinBox, QHBoxLayout, QVBoxLayout, QGridLayout)
from PyQt5.QtGui import QImage, QPixmap, QColor, qRgb, QFont
from PyQt5.QtCore import Qt
style_sheet = """
    QSlider:groove:horizontal{
        border: 1px solid #000000;
        background: white;
        height: 10 px;
        border-radius: 4px
    }
    QSlider#Red:sub-page:horizontal{
        background: qlineargradient(x1: 1, y1: 0, x2: 0, y2: 1,
            stop: 0 #FF4242, stop: 1 #1C1C1C);
        background: qlineargradient(x1: 0, y1: 1, x2: 1, y2: 1,
            stop: 0 #1C1C1C, stop: 1 #FF0000);
        border: 1px solid #4C4B4B;
        height: 10px;
        border-radius: 4px;
    }


    QSlider::add-page:horizontal {
        background: #FFFFFF;
        border: 1px solid #4C4B4B;
        height: 10px;
        border-radius: 4px;
    }
    QSlider::handle:horizontal {
```

```
        background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
            stop:0 #EEEEEE, stop:1 #CCCCCC);
        border: 1px solid #4C4B4B;
        width: 13px;
        margin-top: -3px;
        margin-bottom: -3px;
        border-radius: 4px;
    }
    QSlider::handle:horizontal:hover {
        background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
            stop:0 #FFFFFF, stop:1 #DDDDDD);
        border: 1px solid #393838;
        border-radius: 4px;
    }
    QSlider#Green:sub-page:horizontal{
        background: qlineargradient(x1: 1, y1: 0, x2: 0, y2: 1,
            stop: 0 #FF4242, stop: 1 #1C1C1C);
        background: qlineargradient(x1: 0, y1: 1, x2: 1, y2: 1,
            stop: 0 #1C1C1C, stop: 1 #00FF00);
        border: 1px solid #4C4B4B;
        height: 10px;
        border-radius: 4px;
    }
    QSlider#Blue:sub-page:horizontal{
        background: qlineargradient(x1: 1, y1: 0, x2: 0, y2: 1,
            stop: 0 #FF4242, stop: 1 #1C1C1C);
        background: qlineargradient(x1: 0, y1: 1, x2: 1, y2: 1,
            stop: 0 #1C1C1C, stop: 1 #0000FF);
        border: 1px solid #4C4B4B



;
        height: 10px;
        border-radius: 4px;
    }
"""
class RGBSlider(QWidget):
    def __init__(self, _image=None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._image = _image
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setMinimumSize(225, 300)
        self.setWindowTitle('9.3 - RGB Slider')
        # Store the current pixel value
        self.current_val = QColor()
        self.setupWidgets()
        self.show()
    def setupWidgets(self):
        """



        Create instances of widgets and arrange them in layouts.
        """
        # Image that will display the current color set by
        # slider/spin_box values
        self.color_display = QImage(100, 100, QImage.Format_RGBX64)
        self.color_display.fill(Qt.black)
```

```
        self.cd_label = QLabel()
        self.cd_label.setPixmap(QPixmap.fromImage(self.color_display))
        self.cd_label.setScaledContents(True)
        # Create RGB sliders and spin boxes
        red_label = QLabel("Red")
        red_label.setFont(QFont('Helvetica', 14))
        self.red_slider = QSlider(Qt.Horizontal)
        self.red_slider.setObjectName("Red")
        self.red_slider.setMaximum(255)
        self.red_spinbox = QSpinBox()
        self.red_spinbox.setMaximum(255)
        green_label = QLabel("Green")
        green_label.setFont(QFont('Helvetica', 14))
        self.green_slider = QSlider(Qt.Horizontal)
        self.green_slider.setObjectName("Green")
        self.green_slider.setMaximum(255)
        self.green_spinbox = QSpinBox()
        self.green_spinbox.setMaximum(255)
        blue_label = QLabel("Blue")
        blue_label.setFont(QFont('Helvetica', 14))
        self.blue_slider = QSlider(Qt.Horizontal)
        self.blue_slider.setObjectName("Blue")
        self.blue_slider.setMaximum(255)
        self.blue_spinbox = QSpinBox()
        self.blue_spinbox.setMaximum(255)
        # Use the hex labels to display color values in hex format
        hex_label = QLabel("Hex Color ")
        self.hex_values_label = QLabel()
        hex_h_box = QHBoxLayout()
        hex_h_box.addWidget(hex_label, Qt.AlignRight)
        hex_h_box.addWidget(self.hex_values_label, Qt.AlignRight)
        hex_container = QWidget()
        hex_container.setLayout(hex_h_box)
        # Create grid layout for sliders and spin boxes


        grid = QGridLayout()
        grid.addWidget(red_label, 0, 0, Qt.AlignLeft)
        grid.addWidget(self.red_slider, 1, 0)
        grid.addWidget(self.red_spinbox, 1, 1)
        grid.addWidget(green_label, 2, 0, Qt.AlignLeft)
        grid.addWidget(self.green_slider, 3, 0)
        grid.addWidget(self.green_spinbox, 3, 1)
        grid.addWidget(blue_label, 4, 0, Qt.AlignLeft)
        grid.addWidget(self.blue_slider, 5, 0)
        grid.addWidget(self.blue_spinbox, 5, 1)
        grid.addWidget(hex_container, 6, 0, 1, 0)
        # Use [] to pass arguments to the valueChanged signal
        # The sliders and spin boxes for each color should display the same values and be updated at th
        self.red_slider.valueChanged['int'].connect(self.updateRedSpinBox)
        self.red_spinbox.valueChanged['int'].connect(self.updateRedSlider)
        self.green_slider.valueChanged['int'].connect(self.updateGreenSpinBox)
        self.green_spinbox.valueChanged['int'].connect(self.updateGreenSlider)
        self.blue_slider.valueChanged['int'].connect(self.updateBlueSpinBox)
        self.blue_spinbox.valueChanged['int'].connect(self.updateBlueSlider)
        # Create container for rgb widgets
        rgb_widgets = QWidget()
        rgb_widgets.setLayout(grid)
        v_box = QVBoxLayout()
        v_box.addWidget(self.cd_label)
        v_box.addWidget(rgb_widgets)
```

```
        self.setLayout(v_box)
    # The following methods update the red, green, and blue
    # sliders and spin boxes.
    def updateRedSpinBox(self, value):
        self.red_spinbox.setValue(value)
        self.redValue(value)
    def updateRedSlider(self, value):
        self.red_slider.setValue(value)
        self.redValue(value)
    def updateGreenSpinBox(self, value):
        self.green_spinbox.setValue(value)
        self.greenValue(value)



    def updateGreenSlider(self, value):
        self.green_slider.setValue(value)
        self.greenValue(value)
    def updateBlueSpinBox(self, value):
        self.blue_spinbox.setValue(value)
        self.blueValue(value)
    def updateBlueSlider(self, value):
        self.blue_slider.setValue(value)
        self.blueValue(value)
    # Create new colors based upon the changes to the RGB values
    def redValue(self, value):
        new_color = qRgb(value, self.current_val.green(), self.current_val.blue())
        self.updateColorInfo(new_color)
    def greenValue(self, value):
        new_color = qRgb(self.current_val.red(), value, self.current_val.blue())
        self.updateColorInfo(new_color)
    def blueValue(self, value):
        new_color = qRgb(self.current_val.red(), self.current_val.green(), value)
        self.updateColorInfo(new_color)
    def updateColorInfo(self, color):
        """
        Update color displayed in image and set the hex values accordingly.
        """
        self.current_val = QColor(color)
        self.color_display.fill(color)
        self.cd_label.setPixmap(QPixmap.fromImage(self.color_display))
        self.hex_values_label.setText("{}".format(self.current_val.name()))
    def getPixelValues(self, event):


        """
        The method reimplements the mousePressEvent method.
To use, set a widget's mousePressEvent equal to getPixelValues, like so:
            image_label.mousePressEvent = rgb_slider.getPixelValues
        If an _image != None, then the user can select pixels in the images, and update the sliders to
        rgb and hex values.
        """
        x = event.x()
        y = event.y()
        # valid() returns true if the point selected is a valid
        # coordinate pair within the image
        if self._image.valid(x, y):
            self.current_val = QColor(self._image.pixel(x, y))
            red_val = self.current_val.red()
            green_val = self.current_val.green()
            blue_val = self.current_val.blue()
```

```
            self.updateRedSpinBox(red_val)
            self.updateRedSlider(red_val)
            self.updateGreenSpinBox(green_val)
            self.updateGreenSlider(green_val)
            self.updateBlueSpinBox(blue_val)
            self.updateBlueSlider(blue_val)
```

Listing 9-4 Code for the RGB slider custom widget

An example of the stand-alone widget can be seen in Figure 9-9.

Explanation

We need to import quite a few classes. One worth noting, `qRgb`, is actually a typedef that creates an unsigned int representing the RGB value triplet (r, g, b).

The style sheet that follows the imports is used for changing the appearance of the sliders. We want to modify their appearance so that they give the user more feedback about which RGB values are being changed. Each slider is given an ID selector using the `setObjectName()` method . If no ID selector is used in the style sheet, then that style is applied to all of the `QSlider` objects. The sliders use linear gradients so that users can get a visual representation of how much of the red, green, and blue colors are being used. Refer back to Chapter 6 for a refresher about style sheets.

The RGBSlider class inherits from QWidget. For this class, the user can pass an image and other arguments as parameters in the constructor.

```
class RGBSlider(QWidget):
    def __init__(self, _image=None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._image = _image
```

In `setupWidgets()`, a `QImage` object is created that will display the color created from the RGB values. To display the image in the widget, convert the `QImage` to a `QPixmap` using

```
self.cd_label.setPixmap(QPixmap.fromImage(self.color_display))
```

The contents of the label are then scaled to fit the window's size.

Next, we create each of the red, green, and blue `QSlider` and `QSpinBox` widgets and the labels for displaying the hexadecimal value. The sliders' maximum values are set to `255`, since RGB values are in the range of 0–255. These widgets are then arranged using `QGridLayout`.

Updating the Sliders and Spin Boxes

QSlider and QSpinBox can both emit the `valueChanged()` signal. We can connect the sliders and spin boxes so that their values change relative to each other. For example, when the `red_slider` emits a signal, it triggers the `updateRedSpinBox()` slot, which then updates the `red_spinbox value` using `setValue()`. A similar process happens for the `red_spinbox` and for the green and blue sliders and spin boxes.

```
red_slider.valueChanged['int'].connect(self.updateRedSpinBox)        red_spinbox.valueChanged['int'].co
def updateRedSpinBox(self, value):
    self.red_spinbox.setValue(value)
    self.redValue(value)
```

These widgets are contained in the `rgb_widgets QWidget`. The last thing to do is to arrange the widgets in the main layout.

Updating the Colors

When a signal triggers a slot, it uses `value` to update the corresponding slider or spin box and then calls a function that will create a new color from the red, green, or blue values. The `redValue()` function shown in the following code creates a new `qRgb` color, using the new red `value` and the `current_val`'s `green()` and `blue()` colors. `current_val` is an instance of `QColor`. The QColor class has functions that we can use to access an image's RGB (or other color formats) value.

```
def redValue(self, value):
        new_color = qRgb(value, self.current_val.green(), self.current_val.blue())
        self.updateColorInfo(new_color)
```

The `new_color` is then passed to `updateColorInfo()`. Green and blue colors are handled in a similar fashion. Next we have to create a `QColor` from the `qRgb` value and store it in `current_val`. The `QImage color_display` is updated with `fill()`, which is then converted to a `QPixmap` and displayed on the `cd_label`.

The last thing to do is to update the hexadecimal labels using `QColor.name()`. This function returns the name of the color in the format "#RRGGBB".

Adding Methods to a Custom Widget

The options for methods that you could create for a custom widget are numerous. One option is to create methods that allow the user to modify the behavior or appearance of your custom widget. Another option is to use the event handlers to check for keyboard or mouse events that could be used to interact with your GUI.

`getPixelValue()` is a reimplementation of the `mousePressEvent()` event handler. If an image is passed into the `RGBSlider` constructor, then `_image` is not `None`, and the user can click points in the image to get their corresponding pixel values. `QColor.pixel()` gets a pixel's RGB values. Then, we update `current_val` to use the selected pixel's red, blue, and green values. These values are then passed back into the functions that will update the sliders, spin boxes, labels, and `QImage`.

The following example demonstrates how to implement the color selecting feature.

RGB Slider Demo

One reason for creating a custom widget is so that it can be used in other applications. The following program is a short example of how to import and set up the RGB slider shown in Project 9.3. For this example, an image is displayed in the window alongside the RGB slider. Users can click points within the image and see the RGB and hexadecimal values change in real time.

This short program's GUI can be seen in Figure 9-10.

```
# rgb_demo.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel,
    QHBoxLayout)
from PyQt5.QtGui import QPixmap, QImage
from PyQt5.QtCore import Qt
from rgb_slider import RGBSlider, style_sheet
class ImageDemo(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setMinimumSize(225, 300)
        self.setWindowTitle('9.3 - Custom Widget')
        # Load image
        image = QImage("images/chameleon.png")
        # Create instance of RGB slider widget and pass the image as an argument to RGBSlider
        rgb_slider = RGBSlider(image)
        image_label = QLabel()
        image_label.setAlignment(Qt.AlignTop)
        image_label.setPixmap(QPixmap().fromImage(image))
        # Reimplement the label's mousePressEvent
        image_label.mousePressEvent = rgb_slider.getPixelValues
        h_box = QHBoxLayout()
        h_box.addWidget(rgb_slider)
        h_box.addWidget(image_label)
        self.setLayout(h_box)
        self.show()
if __name__ == '__main__':
    app = QApplication(sys.argv)
    # Use the style_sheet from rgb_slider
    app.setStyleSheet(style_sheet)
    window = ImageDemo()
    sys.exit(app.exec_())
```

Listing 9-5 Code that shows an example for using the RGB slider widget

The RGB slider is a general widget and can be imported into different types of programs. An example can be seen in Figure 9-10.

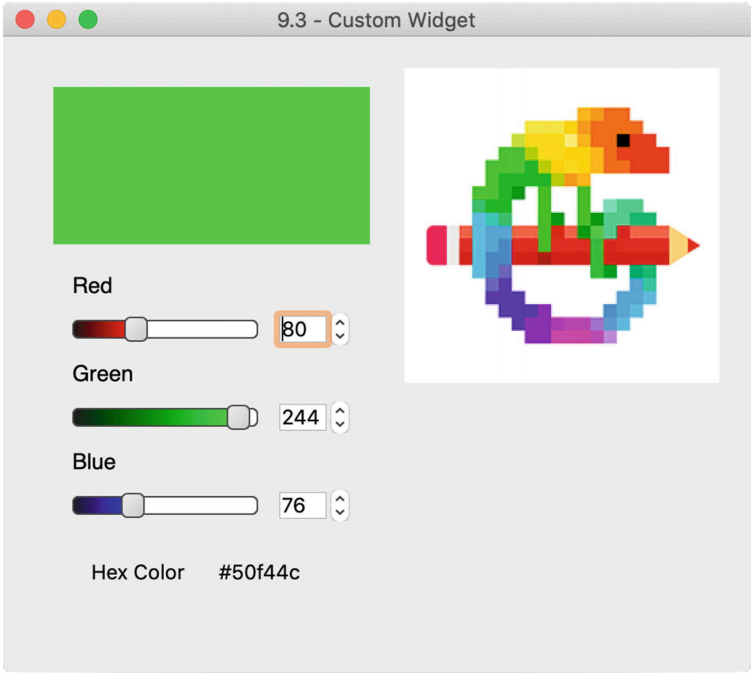

Figure 9-10 An example of including the custom RGB slider in an application

Explanation

Let's get started by importing the classes we need, including the RGB slider and style sheet from `rgb_slider.py`.

```
from rgb_slider import RGBSlider, style_sheet
```

In the `ImageDemo` class , set up the window, create an instance of the RGB slider, and load an image. For this application, we are still creating the image as an instance of `QImage` and then converting it to a `QPixmap`. QImage is used so that we can get access to the image's pixel information.

If you only want to use the slider to get different RGB or hexadecimal values, then the application is finished. Or you could add other functionality to the RGB slider to use in your own projects.

However, we could also reimplement the `QLabel` object's mouse event handler. When the mouse is clicked over a point in the label, we can use the `x` and `y` coordinates from the `event` to update the values in the RGB slider widget using the `RGBSlider` class's `getPixelValues()` method .

```
image_label.mousePressEvent = rgb_slider.getPixelValues
```

Summary

PyQt5's graphics and painting system is an extensive topic that could be an entire book by itself. The QPainter class is important for performing the painting on widgets and on other paint devices. QPainter works together with the QPaintEngine and QPaintDevice classes to provide the tools you need for creating two-dimensional drawing applications.

In Chapter 9, we have taken a look at some of QPainter's functions for drawing lines, primitive and abstract shapes. Together with QPen,

QBrush, and QColor, QPainter is able to create some rather beautiful digital images. To materialize this concept, we created a simple painting application. Hopefully, you use that application and add even more drawing features.

We also saw how to create properties for objects made from the QObject class and then animate those objects in Qt Graphics View Framework. It is not covered in this book, but you could use the Graphics View to create a GUI with items that are interactive.

Finally, one of PyQt's strengths comes from being able to customize the built-in widgets or to create your own widget that can then be imported seamlessly into other applications.

In Chapter **10**, we will learn about data handling using databases and PyQt.

Footnotes

1 https://doc.qt.io/qt-5/qtwidgets-widgets-scribble-example.html